

M13 - Delta Lake + Lakehouse

11-lesson learner book | Beginner-first professional institute standard | B01-style teaching contract

Purpose

Operate Delta/lakehouse tables with ACID, MERGE, schema evolution, history and layer contracts.

Course boundary

This is the independent Standalone Data Engineer course, not Four-Course C3. Complete the course in sequence; each new idea is still taught from first principles.

Option B institute identity | Charcoal #1F2937 | Teal #14B8A6 | Soft Blue #3B82F6

This book is your teacher

Use one lesson at a time. Do not read protected review before saving a genuine attempt. Every lesson follows the same beginner-first sequence even when the engineering idea becomes more technical.

Daily lesson loop

1) why it matters -> 2) what you learn -> 3) prerequisite -> 4) picture it -> 5) new words -> 6) useful visual/source -> 7) follow once -> 8) expected result -> 9) why it worked -> 10) mistakes -> 11) recovery -> 12) independent task -> 13) validate -> 14) teach it back -> 15) protected review + changed retry -> 16) evidence/mastery + exact next route.

Prerequisite check

Complete M11 storage/file-format foundations and M12 Spark/PySpark basics. You should understand Parquet, partitions, schemas, and distributed reads/writes.

Attempt protection

A protected review is repair material, not a shortcut. Save the first attempt, open only the matching review if needed, record the repair, close review, then solve a changed retry from a clean state.

Contents / Index

Every formal lesson and its actual page number in this exact rebuilt book.

DE13-01 - Data Lake vs Warehouse vs Lakehouse	5
DE13-02 - Why plain files are not tables	7
DE13-03 - Transaction log, ACID and table versions	9
DE13-04 - Create, read, update and delete	11
DE13-05 - Schema enforcement and schema evolution	13
DE13-06 - Time travel and version history	15
DE13-07 - MERGE, upserts and idempotent incremental loads	17
DE13-08 - Slowly Changing Dimension Type 2 with Delta	19
DE13-09 - Bronze, Silver and Gold layer contracts	21
DE13-10 - File layout, compaction and table maintenance	23
DE13-11 - Lakehouse integration and debugging	25

Prerequisite, tool and outcome map

ASSUMED

Complete M11 storage/file-format foundations and M12 Spark/PySpark basics. You should understand Parquet, partitions, schemas, and distributed reads/writes.

MODULE PURPOSE

Operate Delta/lakehouse tables with ACID, MERGE, schema evolution, history and layer contracts.

EXIT EVIDENCE

Module practical + fresh retest + saved evidence; no critical unresolved failure.

Protected study route

Teaching -> assigned visual/source when useful -> follow once -> close worked material -> independent attempt -> validate -> save evidence -> teach it back. If weak: matching protected review -> targeted repair -> close review -> changed fresh retry -> revalidate.

DE13-01 - Data Lake vs Warehouse vs Lakehouse

1. Why it matters

By the end of this lesson you can data lake vs warehouse vs lakehouse and prove the result with saved evidence rather than a plausible screenshot. This matters because lakehouse tables depend on transaction-log state and version history; treating them as loose files can break reproducibility and recovery.

2. What you are learning

By the end of this lesson you can data lake vs warehouse vs lakehouse and prove the result with saved evidence rather than a plausible screenshot.

3. Prerequisite / what you already know

Complete M11 storage/file-format foundations and M12 Spark/PySpark basics. You should understand Parquet, partitions, schemas, and distributed reads/writes.

4. Picture it in your head

A lake emphasizes flexible files, a warehouse emphasizes governed relational analytics, and a lakehouse adds table/transaction semantics over lake-style storage.

5. New words before we use them

A data lake stores flexible files cheaply; a warehouse provides managed tables and query-oriented governance; a lakehouse tries to combine open/low-cost storage with table semantics and analytical management. The useful question is not which label sounds modern. Ask what guarantees the consumer needs: transactions, schema, history, reproducibility, access control, performance and tooling.

6. Useful video / source

Source: Databricks — Diving into Delta Lake: Transaction Log

Assigned scope: Use matching transaction-log / ACID / time-travel section

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=IXIBJdehdsdw)

URL: <https://www.youtube.com/watch?v=IXIBJdehdsdw>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Draw three boxes: lake, warehouse, lakehouse. For each, write storage form, table guarantees, typical consumers, and one trade-off. Then place Parquet and Delta correctly: Parquet is a file format; Delta adds table metadata/transaction semantics around data files. Classify five scenarios as lake, warehouse, or lakehouse and justify by guarantees, not brand names.

8. Expected result / what you should see

You should be able to by the end of this lesson you can data lake vs warehouse vs lakehouse and prove the result with saved evidence rather than a plausible screenshot and show a concrete result/state that another person can inspect. Observable target: For Data Lake vs Warehouse vs Lakehouse: Check current row/key/totals plus Delta history/version.

9. Why it worked

The table operation is trustworthy because the transaction log defines one committed table state; version/history and row controls prove the logical result rather than trusting physical files alone. Your independent check is: Do not validate Data Lake vs Warehouse vs Lakehouse by appearance alone. The proof is committed table-version history plus row/key reconciliation.

10. Common beginner mistakes

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup. DE13-01 - Practice and proof

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A lake emphasizes flexible files, a warehouse emphasizes governed relational analytics, and a lakehouse adds table/transaction semantics over lake-style storage. 3. Run the earliest independent control: Do not validate Data Lake vs Warehouse vs Lakehouse by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

For an IoT telemetry workload, choose lake/warehouse/lakehouse and defend the choice using transactions, history and analytical access.

13. Validate your result

Do not validate Data Lake vs Warehouse vs Lakehouse by appearance alone. Use the lesson-specific control: Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Data Lake vs Warehouse vs Lakehouse in your own words. Start with the purpose: By the end of this lesson you can data lake vs warehouse vs lakehouse and prove the result with saved evidence rather than a plausible screenshot. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Data Lake vs Warehouse vs Lakehouse by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Data Lake vs Warehouse vs Lakehouse with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE13-02.

DE13-02 - Why plain files are not tables

1. Why it matters

By the end of this lesson you can why plain files are not tables and prove the result with saved evidence rather than a plausible screenshot. This matters because lakehouse tables depend on transaction-log state and version history; treating them as loose files can break reproducibility and recovery.

2. What you are learning

By the end of this lesson you can why plain files are not tables and prove the result with saved evidence rather than a plausible screenshot.

3. Prerequisite / what you already know

Prerequisite: DE13-01 mastered with saved evidence.

4. Picture it in your head

A folder of Parquet files has bytes and schemas but no single authoritative transaction history. A table adds metadata/log rules that define one committed state.

5. New words before we use them

A directory full of Parquet files can hold typed columnar data, but the files alone do not tell concurrent writers which set of files forms the committed table snapshot. A table layer needs metadata that says which files are active, what schema is valid, and which operation produced the current state. That is why "Parquet folder" and "Delta table" are not interchangeable.

6. Useful video / source

Source: Databricks — Diving into Delta Lake: Transaction Log

Assigned scope: Use matching transaction-log / ACID / time-travel section

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE ([youtube.com](https://www.youtube.com/watch?v=IXIBJdehsdw))

URL: <https://www.youtube.com/watch?v=IXIBJdehsdw>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Use the normal simulator output. Compare version_0.json with the input CSV files. Identify the extra table-state information: version, operation, schema and active rows. Explain why a consumer needs a committed snapshot rather than a random directory listing. Inspect base files and version_0 table snapshot; list three guarantees the snapshot metadata adds.

8. Expected result / what you should see

You should be able to by the end of this lesson you can why plain files are not tables and prove the result with saved evidence rather than a plausible screenshot and show a concrete result/state that another person can inspect. Observable target: For Why plain files are not tables: Verify the exact path, filename/extension, byte size or hash where appropriate, and confirm the original/working-copy rule was followed.

9. Why it worked

The spreadsheet result is trustworthy because the source structure/formula logic is preserved and an independent cell/row/total check proves the displayed result came from the intended calculation. Your independent check is: Do not validate Why plain files are not tables by appearance alone. The proof is committed table-version history plus row/key reconciliation.

10. Common beginner mistakes

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup. DE13-02 - Practice and proof

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A folder of Parquet files has bytes and schemas but no single authoritative transaction history. 3. Run the earliest independent control: Do not validate Why plain files are not tables by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Explain why copying a new Parquet file into a table directory is not equivalent to a committed table write.

13. Validate your result

Do not validate Why plain files are not tables by appearance alone. Use the lesson-specific control: Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Why plain files are not tables in your own words. Start with the purpose: By the end of this lesson you can why plain files are not tables and prove the result with saved evidence rather than a plausible screenshot. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Why plain files are not tables by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Why plain files are not tables with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE13-03.

DE13-03 - Transaction log, ACID and table versions

1. Why it matters

By the end of this lesson you can transaction log, acid and table versions and prove the result with saved evidence rather than a plausible screenshot. This matters because lakehouse tables depend on transaction-log state and version history; treating them as loose files can break reproducibility and recovery.

2. What you are learning

By the end of this lesson you can transaction log, acid and table versions and prove the result with saved evidence rather than a plausible screenshot.

3. Prerequisite / what you already know

Prerequisite: DE13-02 mastered with saved evidence.

4. Picture it in your head

Git has three useful areas: working tree -> staging area -> commit history. status tells where changes are; add selects the snapshot; commit records it.

5. New words before we use them

Delta records table changes in an ordered transaction log. A successful commit creates a new version; readers reconstruct a consistent snapshot from committed actions rather than seeing a half-finished multi-file write. ACID is about transaction behavior, not about making bad business logic correct.

6. Useful video / source

Source: Databricks — Diving into Delta Lake: Transaction Log

Assigned scope: Use matching transaction-log / ACID / time-travel section

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE (youtube.com)

URL: <https://www.youtube.com/watch?v=IXIBJdehdsdw>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Open normal/log. Read versions 0 through 3. For each, state operation and row/value control. Show that version numbers are immutable history in the simulator. Then explain what real Delta would add beyond this classroom JSON model. Read commit logs 0–3 and explain atomic committed-version visibility.

8. Expected result / what you should see

You should be able to by the end of this lesson you can transaction log, acid and table versions and prove the result with saved evidence rather than a plausible screenshot and show a concrete result/state that another person can inspect. Observable target: For Transaction log, ACID and table versions: Capture before-state, execute the transaction, verify committed rows/totals, then test rollback or replay behavior on a changed case.

9. Why it worked

The operation is trustworthy because related changes cross the commit boundary together; a before/after state check proves partial updates are not being mistaken for success. Your independent check is: Do not validate Transaction log, ACID and table versions by appearance alone. The proof is committed table-version history plus row/key reconciliation.

10. Common beginner mistakes

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup. DE13-03 - Practice and proof

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Git has three useful areas: working tree -> staging area -> commit history. 3. Run the earliest independent control: Do not validate Transaction log, ACID and table versions by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Given four hypothetical commits, identify what a reader at version 2 should see and what it must not see.

13. Validate your result

Do not validate Transaction log, ACID and table versions by appearance alone. Use the lesson-specific control: Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Transaction log, ACID and table versions in your own words. Start with the purpose: By the end of this lesson you can transaction log, acid and table versions and prove the result with saved evidence rather than a plausible screenshot. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Transaction log, ACID and table versions by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Transaction log, ACID and table versions with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE13-04.

DE13-04 - Create, read, update and delete

1. Why it matters

By the end of this lesson you can create, read, update and delete and prove the result with saved evidence rather than a plausible screenshot. This matters because lakehouse tables depend on transaction-log state and version history; treating them as loose files can break reproducibility and recovery.

2. What you are learning

By the end of this lesson you can create, read, update and delete and prove the result with saved evidence rather than a plausible screenshot.

3. Prerequisite / what you already know

Prerequisite: DE13-03 mastered with saved evidence.

4. Picture it in your head

Treat Delta DML as table-level transactions: each write produces a new committed version, while readers see a consistent snapshot rather than half-written files.

5. New words before we use them

CRUD describes logical table changes: create/write an initial table, read a snapshot, update matched rows, and delete rows intentionally. In Delta, updates/deletes create new table versions rather than editing a Parquet file in place as if it were a spreadsheet. The transaction layer chooses new active files and records the commit.

6. Useful video / source

Source: Databricks Academy — free Delta / Lakehouse training

Assigned scope: Choose the matching Delta/Lakehouse unit

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE (docs.databricks.com)

URL: <https://docs.databricks.com/aws/en/getting-started/free-training>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Trace $v_0 \rightarrow v_1 \rightarrow v_2 \rightarrow v_3$ in the normal simulator. Identify the changed and deleted business IDs. Then write pseudocode for CREATE, UPDATE and DELETE in Delta SQL or DeltaTable API without copying the reference implementation.

Trace create/read/update/delete transitions and reconcile each version row/value controls.

8. Expected result / what you should see

You should be able to by the end of this lesson you can create, read, update and delete and prove the result with saved evidence rather than a plausible screenshot and show a concrete result/state that another person can inspect.

Observable target: For Create, read, update and delete: record the actual before/after state and verify the lesson requirement with an independent control.

9. Why it worked

This works because by the end of this lesson you can create, read, update and delete and prove the result with saved evidence rather than a plausible screenshot is made explicit before the result is accepted. Your independent check is: Do not validate Create, read, update and delete by appearance alone. The proof is committed table-version history plus row/key reconciliation.

10. Common beginner mistakes

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup. DE13-04 - Practice and proof

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Treat Delta DML as table-level transactions: each write produces a new committed version, while readers see a consistent snapshot rather than half-written files. 3. Run the earliest independent control: Do not validate Create, read, update and delete by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Write your own CRUD pseudocode and the validation after each operation.

13. Validate your result

Do not validate Create, read, update and delete by appearance alone. Use the lesson-specific control: Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Create, read, update and delete in your own words. Start with the purpose: By the end of this lesson you can create, read, update and delete and prove the result with saved evidence rather than a plausible screenshot. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Create, read, update and delete by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Create, read, update and delete with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE13-05.

DE13-05 - Schema enforcement and schema evolution

1. Why it matters

By the end of this lesson you can schema enforcement and schema evolution and prove the result with saved evidence rather than a plausible screenshot. This matters because lakehouse tables depend on transaction-log state and version history; treating them as loose files can break reproducibility and recovery.

2. What you are learning

By the end of this lesson you can schema enforcement and schema evolution and prove the result with saved evidence rather than a plausible screenshot.

3. Prerequisite / what you already know

Prerequisite: DE13-04 mastered with saved evidence.

4. Picture it in your head

Schema enforcement blocks unexpected shape; schema evolution changes the contract deliberately. Picture a guarded doorway whose allowed shape can change only through an approved rule.

5. New words before we use them

Schema enforcement rejects incompatible writes instead of quietly corrupting a table. Schema evolution is a controlled decision to accept a compatible structural change. "Automatically accept every schema change" is not resilience.

6. Useful video / source

Source: Databricks — Diving into Delta Lake: Transaction Log

Assigned scope: Use matching transaction-log / ACID / time-travel section

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=XIBJdehdsdw)

URL: <https://www.youtube.com/watch?v=XIBJdehdsdw>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Compare v1 schema with v2 schema. Explain why adding `quality_band` is a controlled additive change. Then propose one breaking change—such as value becoming free-text—and write the validation that should stop it. Apply one additive schema change and reject one incompatible type-change proposal with a written reason.

8. Expected result / what you should see

You should be able to by the end of this lesson you can schema enforcement and schema evolution and prove the result with saved evidence rather than a plausible screenshot and show a concrete result/state that another person can inspect. Observable target: For Schema enforcement and schema evolution: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced.

9. Why it worked

The result is trustworthy because structure is treated as a contract rather than inferred casually; the validation checks fields/types/constraints against the declared shape. Your independent check is: Do not validate Schema enforcement and schema evolution by appearance alone. The proof is committed table-version history plus row/key reconciliation.

10. Common beginner mistakes

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup. DE13-05 - Practice and proof

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Schema enforcement blocks unexpected shape; schema evolution changes the contract deliberately. 3. Run the earliest independent control: Do not validate Schema enforcement and schema evolution by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Classify four schema changes as safe additive, controlled migration, or reject; explain downstream impact.

13. Validate your result

Do not validate Schema enforcement and schema evolution by appearance alone. Use the lesson-specific control: Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Schema enforcement and schema evolution in your own words. Start with the purpose: By the end of this lesson you can schema enforcement and schema evolution and prove the result with saved evidence rather than a plausible screenshot. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Schema enforcement and schema evolution by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Schema enforcement and schema evolution with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE13-06.

DE13-06 - Time travel and version history

1. Why it matters

By the end of this lesson you can time travel and version history and prove the result with saved evidence rather than a plausible screenshot. This matters because lakehouse tables depend on transaction-log state and version history; treating them as loose files can break reproducibility and recovery.

2. What you are learning

By the end of this lesson you can time travel and version history and prove the result with saved evidence rather than a plausible screenshot.

3. Prerequisite / what you already know

Prerequisite: DE13-05 mastered with saved evidence.

4. Picture it in your head

A pipe connects one command's output to the next command's input; redirection sends output to a file. Picture commands as small boxes connected by arrows.

5. New words before we use them

Time travel means reading an earlier committed table version or timestamp while its required files are still retained. History is useful for auditing, debugging and recovery, but it is not a forever backup. VACUUM/retention policies can make old snapshots unreadable after old files are physically removed.

6. Useful video / source

Source: Databricks — Diving into Delta Lake: Transaction Log

Assigned scope: Use matching transaction-log / ACID / time-travel section

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE (youtube.com)

URL: <https://www.youtube.com/watch?v=IXIBJdehsw>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Read normal version 0 and current version 2. Compare Ravi/C102 tier and total value. Write a debugging note explaining which version introduced the change and what evidence you used. Compare v0 and v2, identify the change version, and state the retention dependency of time travel.

8. Expected result / what you should see

You should be able to by the end of this lesson you can time travel and version history and prove the result with saved evidence rather than a plausible screenshot and show a concrete result/state that another person can inspect.

Observable target: For Time travel and version history: Check current row/key/totals plus Delta history/version.

9. Why it worked

The table operation is trustworthy because the transaction log defines one committed table state; version/history and row controls prove the logical result rather than trusting physical files alone. Your independent check is: Do not validate Time travel and version history by appearance alone. The proof is committed table-version history plus row/key reconciliation.

10. Common beginner mistakes

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup. DE13-06 - Practice and proof

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A pipe connects one command's output to the next command's input; redirection sends output to a file. 3. Run the earliest independent control: Do not validate Time travel and version history by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Write a recovery plan using history/time travel without claiming it replaces backup.

13. Validate your result

Do not validate Time travel and version history by appearance alone. Use the lesson-specific control: Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Time travel and version history in your own words. Start with the purpose: By the end of this lesson you can time travel and version history and prove the result with saved evidence rather than a plausible screenshot. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Time travel and version history by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Time travel and version history with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE13-07.

DE13-07 - MERGE, upserts and idempotent incremental loads

1. Why it matters

By the end of this lesson you can merge, upserts and idempotent incremental loads and prove the result with saved evidence rather than a plausible screenshot. This matters because lakehouse tables depend on transaction-log state and version history; treating them as loose files can break reproducibility and recovery.

2. What you are learning

By the end of this lesson you can merge, upserts and idempotent incremental loads and prove the result with saved evidence rather than a plausible screenshot.

3. Prerequisite / what you already know

Prerequisite: DE13-06 mastered with saved evidence.

4. Picture it in your head

A transaction groups related inserts/updates/deletes into one atomic unit. UPSERT adds “insert if new, update if existing” but still needs a trustworthy conflict key.

5. New words before we use them

MERGE combines source changes with a target using a match key. A common upsert rule is: matched key → update; unseen key → insert. Rerun safety depends on deterministic match logic and source uniqueness.

6. Useful video / source

Source: Databricks Academy — free Delta / Lakehouse training

Assigned scope: Choose the matching Delta/Lakehouse unit

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE (docs.databricks.com)

URL: <https://docs.databricks.com/aws/en/getting-started/free-training>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Use updates.csv and prove one updated ID plus one inserted ID. Run the simulator twice and prove the second run does not add a sixth row. Then inspect duplicate_source.csv and explain why source-key uniqueness is a precondition. Run normal MERGE simulator, rerun it, and prove row/value idempotency; then trigger duplicate-source diagnosis.

8. Expected result / what you should see

You should be able to by the end of this lesson you can merge, upserts and idempotent incremental loads and prove the result with saved evidence rather than a plausible screenshot and show a concrete result/state that another person can inspect. Observable target: For MERGE, upserts and idempotent incremental loads: Check pre/post row counts, key uniqueness on the expected “one” side, unmatched-key counts, and one important total before and after the relationship operation.

9. Why it worked

The workflow is trustworthy because progress/state is explicit and replaying the same input is tested to converge without silent loss or duplicate side effects. Your independent check is: Do not validate MERGE, upserts and idempotent incremental loads by appearance alone. The proof is committed table-version history plus row/key reconciliation.

10. Common beginner mistakes

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup. DE13-07 - Practice and proof

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A transaction groups related inserts/updates/deletes into one atomic unit. 3. Run the earliest independent control: Do not validate MERGE, upserts and idempotent incremental loads by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Design an idempotent upsert contract: key, source uniqueness check, matched action, unmatched action, rerun proof.

13. Validate your result

Do not validate MERGE, upserts and idempotent incremental loads by appearance alone. Use the lesson-specific control: Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain MERGE, upserts and idempotent incremental loads in your own words. Start with the purpose: By the end of this lesson you can merge, upserts and idempotent incremental loads and prove the result with saved evidence rather than a plausible screenshot. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate MERGE, upserts and idempotent incremental loads by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for MERGE, upserts and idempotent incremental loads with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE13-08.

DE13-08 - Slowly Changing Dimension Type 2 with Delta

1. Why it matters

By the end of this lesson you can slowly changing dimension type 2 with delta and prove the result with saved evidence rather than a plausible screenshot. This matters because lakehouse tables depend on transaction-log state and version history; treating them as loose files can break reproducibility and recovery.

2. What you are learning

By the end of this lesson you can slowly changing dimension type 2 with delta and prove the result with saved evidence rather than a plausible screenshot.

3. Prerequisite / what you already know

Prerequisite: DE13-07 mastered with saved evidence.

4. Picture it in your head

SCD Type 2 is a history ledger: when an attribute changes, close the old version and add a new dated version instead of overwriting the past.

5. New words before we use them

SCD Type 2 preserves attribute history instead of overwriting it. When a tracked attribute changes, expire the old dimension row and insert a new current row with a new effective period. Business keys identify the entity across versions; surrogate/version rows represent historical states.

6. Useful video / source

Source: Databricks Academy — free Delta / Lakehouse training

Assigned scope: Choose the matching Delta/Lakehouse unit

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - [CLICK HERE \(docs.databricks.com\)](https://docs.databricks.com)

URL: <https://docs.databricks.com/aws/en/getting-started/free-training>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Open `scd2_history.json`. Find the changed business key. Verify the old row is closed and the new row is current. State two invariants: one current row per key; historical intervals do not overlap. Validate the generated SCD2 history for one current row per business key and non-overlapping periods.

8. Expected result / what you should see

You should be able to by the end of this lesson you can slowly changing dimension type 2 with delta and prove the result with saved evidence rather than a plausible screenshot and show a concrete result/state that another person can inspect. Observable target: For Slowly Changing Dimension Type 2 with Delta: Check current row/key/totals plus Delta history/version.

9. Why it worked

The table operation is trustworthy because the transaction log defines one committed table state; version/history and row controls prove the logical result rather than trusting physical files alone. Your independent check is: Do not validate Slowly Changing Dimension Type 2 with Delta by appearance alone. The proof is committed table-version history plus row/key reconciliation.

10. Common beginner mistakes

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup. DE13-08 - Practice and proof

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: SCD Type 2 is a history ledger: when an attribute changes, close the old version and add a new dated version instead of overwriting the past. 3. Run the earliest independent control: Do not validate Slowly Changing Dimension Type 2 with Delta by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Design SCD2 rows for a customer whose tier changes twice; prove one current row remains.

13. Validate your result

Do not validate Slowly Changing Dimension Type 2 with Delta by appearance alone. Use the lesson-specific control: Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Slowly Changing Dimension Type 2 with Delta in your own words. Start with the purpose: By the end of this lesson you can slowly changing dimension type 2 with delta and prove the result with saved evidence rather than a plausible screenshot. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Slowly Changing Dimension Type 2 with Delta by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Slowly Changing Dimension Type 2 with Delta with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE13-09.

DE13-09 - Bronze, Silver and Gold layer contracts

1. Why it matters

By the end of this lesson you can bronze, silver and gold layer contracts and prove the result with saved evidence rather than a plausible screenshot. This matters because lakehouse tables depend on transaction-log state and version history; treating them as loose files can break reproducibility and recovery.

2. What you are learning

By the end of this lesson you can bronze, silver and gold layer contracts and prove the result with saved evidence rather than a plausible screenshot.

3. Prerequisite / what you already know

Prerequisite: DE13-08 mastered with saved evidence.

4. Picture it in your head

Bronze preserves received data, Silver enforces trusted structure/business rules, and Gold serves consumer-ready models. Each layer has a different contract, not just a different folder name.

5. New words before we use them

Medallion architecture is a design pattern, not three magical folders. Bronze preserves raw/replayable source truth plus ingestion metadata. Silver applies reusable cleaning, typing, deduplication and quality rules at a declared grain.

6. Useful video / source

Source: Databricks Academy — free Delta / Lakehouse training

Assigned scope: Choose the matching Delta/Lakehouse unit

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE (docs.databricks.com)

URL: <https://docs.databricks.com/aws/en/getting-started/free-training>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Use normal controls: compare `bronze_rows`, `silver_rows` and `gold_value`. Explain why Bronze can contain both base and update events while Silver represents current-state records. Reconcile Gold total back to Silver before trusting it. Map raw events to Bronze, current clean state to Silver, and region aggregate to Gold; reconcile totals.

8. Expected result / what you should see

You should be able to by the end of this lesson you can bronze, silver and gold layer contracts and prove the result with saved evidence rather than a plausible screenshot and show a concrete result/state that another person can inspect. Observable target: For Bronze, Silver and Gold layer contracts: record the actual before/after state and verify the lesson requirement with an independent control.

9. Why it worked

The result is trustworthy because structure is treated as a contract rather than inferred casually; the validation checks fields/types/constraints against the declared shape. Your independent check is: Do not validate Bronze, Silver and Gold layer contracts by appearance alone. The proof is committed table-version history plus row/key reconciliation.

10. Common beginner mistakes

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup. DE13-09 - Practice and proof

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Bronze preserves received data, Silver enforces trusted structure/business rules, and Gold serves consumer-ready models. 3. Run the earliest independent control: Do not validate Bronze, Silver and Gold layer contracts by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Define Bronze/Silver/Gold contracts for clinic events, including grain, required fields, quarantine and reconciliation.

13. Validate your result

Do not validate Bronze, Silver and Gold layer contracts by appearance alone. Use the lesson-specific control: Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Bronze, Silver and Gold layer contracts in your own words. Start with the purpose: By the end of this lesson you can bronze, silver and gold layer contracts and prove the result with saved evidence rather than a plausible screenshot. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Bronze, Silver and Gold layer contracts by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Bronze, Silver and Gold layer contracts with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE13-10.

DE13-10 - File layout, compaction and table maintenance

1. Why it matters

By the end of this lesson you can file layout, compaction and table maintenance and prove the result with saved evidence rather than a plausible screenshot. This matters because lakehouse tables depend on transaction-log state and version history; treating them as loose files can break reproducibility and recovery.

2. What you are learning

By the end of this lesson you can file layout, compaction and table maintenance and prove the result with saved evidence rather than a plausible screenshot.

3. Prerequisite / what you already know

Prerequisite: DE13-09 mastered with saved evidence.

4. Picture it in your head

Many tiny files are like thousands of tiny parcels: metadata/open costs dominate. Compaction combines them into healthier file sizes while preserving table semantics.

5. New words before we use them

Logical correctness and physical maintenance are separate. OPTIMIZE/compaction can rewrite many small data files into fewer larger files without changing table meaning. Partition choices should follow access patterns and data size; high-cardinality IDs usually make poor partition columns.

6. Useful video / source

Source: Databricks Academy — free Delta / Lakehouse training

Assigned scope: Choose the matching Delta/Lakehouse unit

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - [CLICK HERE \(docs.databricks.com\)](https://docs.databricks.com)

URL: <https://docs.databricks.com/aws/en/getting-started/free-training>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Write a maintenance plan for a table partitioned by a low-cardinality date/region field. Include: symptom, metric, compaction action, validation before/after, retention rule, and rollback/time-travel impact. Design a compaction/partition/VACUUM maintenance checklist that preserves logical controls.

8. Expected result / what you should see

You should be able to by the end of this lesson you can file layout, compaction and table maintenance and prove the result with saved evidence rather than a plausible screenshot and show a concrete result/state that another person can inspect. Observable target: For File layout, compaction and table maintenance: Verify the exact path, filename/extension, byte size or hash where appropriate, and confirm the original/working-copy rule was followed.

9. Why it worked

The spreadsheet result is trustworthy because the source structure/formula logic is preserved and an independent cell/row/total check proves the displayed result came from the intended calculation. Your independent check is: Do not validate File layout, compaction and table maintenance by appearance alone. The proof is committed table-version history plus row/key reconciliation.

10. Common beginner mistakes

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup. DE13-10 - Practice and proof

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Many tiny files are like thousands of tiny parcels: metadata/open costs dominate. 3. Run the earliest independent control: Do not validate File layout, compaction and table maintenance by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Choose a partition key for a 2-year order table and reject order_id; define compaction and retention checks.

13. Validate your result

Do not validate File layout, compaction and table maintenance by appearance alone. Use the lesson-specific control: Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain File layout, compaction and table maintenance in your own words. Start with the purpose: By the end of this lesson you can file layout, compaction and table maintenance and prove the result with saved evidence rather than a plausible screenshot. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate File layout, compaction and table maintenance by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for File layout, compaction and table maintenance with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE13-11.

DE13-11 - Lakehouse integration and debugging

1. Why it matters

By the end of this lesson you can lakehouse integration and debugging and prove the result with saved evidence rather than a plausible screenshot. This matters because lakehouse tables depend on transaction-log state and version history; treating them as loose files can break reproducibility and recovery.

2. What you are learning

By the end of this lesson you can lakehouse integration and debugging and prove the result with saved evidence rather than a plausible screenshot.

3. Prerequisite / what you already know

Prerequisite: DE13-10 mastered with saved evidence.

4. Picture it in your head

A backfill replays historical input through the same controlled path. Reconciliation compares source receipts with landed/processed totals so missing or duplicated periods are visible.

5. New words before we use them

Debug a lakehouse from contracts outward: source receipt → Bronze landing → Silver validity → Gold reconciliation → transaction/history evidence → physical layout. Do not start by randomly deleting checkpoints or VACUUMing. Use table history and control totals to locate the first bad boundary.

6. Useful video / source

Source: Databricks Academy — free Delta / Lakehouse training

Assigned scope: Choose the matching Delta/Lakehouse unit

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE (docs.databricks.com)

URL: <https://docs.databricks.com/aws/en/getting-started/free-training>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Take the unseen practical dataset. Write a six-step incident runbook that identifies the first bad layer, uses row/value controls, inspects history, checks source-key uniqueness before MERGE, and preserves evidence before repair. Debug a seeded mismatch using layer-by-layer controls and table history without destructive cleanup.

8. Expected result / what you should see

You should be able to by the end of this lesson you can lakehouse integration and debugging and prove the result with saved evidence rather than a plausible screenshot and show a concrete result/state that another person can inspect. Observable target: For Lakehouse integration and debugging: Check current row/key/totals plus Delta history/version.

9. Why it worked

The table operation is trustworthy because the transaction log defines one committed table state; version/history and row controls prove the logical result rather than trusting physical files alone. Your independent check is: Do not validate Lakehouse integration and debugging by appearance alone. The proof is committed table-version history plus row/key reconciliation.

10. Common beginner mistakes

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup. DE13-11 - Practice and proof

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A backfill replays historical input through the same controlled path. 3. Run the earliest independent control: Do not validate Lakehouse integration and debugging by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Diagnose a scenario where Gold revenue drops but Bronze receipt count is normal; state the first three checks in order.

13. Validate your result

Do not validate Lakehouse integration and debugging by appearance alone. Use the lesson-specific control: Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Lakehouse integration and debugging in your own words. Start with the purpose: By the end of this lesson you can lakehouse integration and debugging and prove the result with saved evidence rather than a plausible screenshot. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Lakehouse integration and debugging by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Lakehouse integration and debugging with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: module practical / Gate route after the mastery checklist.